

SDA-Pro Comprehensive Project Report

Security Incident Response & Threat Mitigation Platform

M.Usman (49649)

Farhan ijaz()

Executive Summary

This report documents the implemented SDA-Pro university semester project: a demo-ready Security Incident Response and Threat Mitigation Platform. The system focuses on Software Design and Architecture evidence: design patterns, architecture styles, service boundaries, API contracts, asynchronous events, database schema, frontend screens, UML diagrams, and tests. External cybersecurity products are intentionally mocked so the demo can run locally without real API keys.

Implementation note: the backend features are packaged into a Spring Boot composition root for reliable local demo execution, while the repository, contracts, diagrams, and documentation preserve clear SOA-style service boundaries. This is appropriate for a semester architecture demonstration, but it is not claimed to be a production microservices deployment.

Final Tech Stack

Layer	Technology	Proof
Backend	Java 21, Spring Boot 3, Maven	services/alert-ingestion-service
Frontend	React, TypeScript, Vite	soc-dashboard
Database	PostgreSQL 16	docker-compose service postgres
Cache	Redis 7	Threat intelligence proxy cache
Message broker	RabbitMQ 3	Event publishing/subscribing
Realtime	Server-Sent Events dashboard stream	/dashboard/events
Docs/UML	PlantUML and ADR markdown	docs/uml diagrams, docs/adr
Deployment	Docker Compose	docker-compose.yml

System Architecture

The project is organized as a local SOA demonstration. Alerts enter through ingestion APIs, are normalized, enriched, correlated into incidents, advanced through lifecycle states, responded to using strategy-selected actions, published as events, pushed to the dashboard, and written to audit storage.

- Controllers expose required API endpoints and route requests into application services.
- Application services coordinate normalization, enrichment, correlation, state transitions, response execution, notification, and audit.
- Domain classes contain incident states, alert composites, response strategies, and event models.
- Adapters and proxies isolate mock Splunk, Firewall, VirusTotal, MISP, Slack, PagerDuty, Email, and Active Directory behavior.
- PostgreSQL stores durable records, Redis caches threat intelligence, RabbitMQ represents event-driven communication, and SSE updates the SOC dashboard.

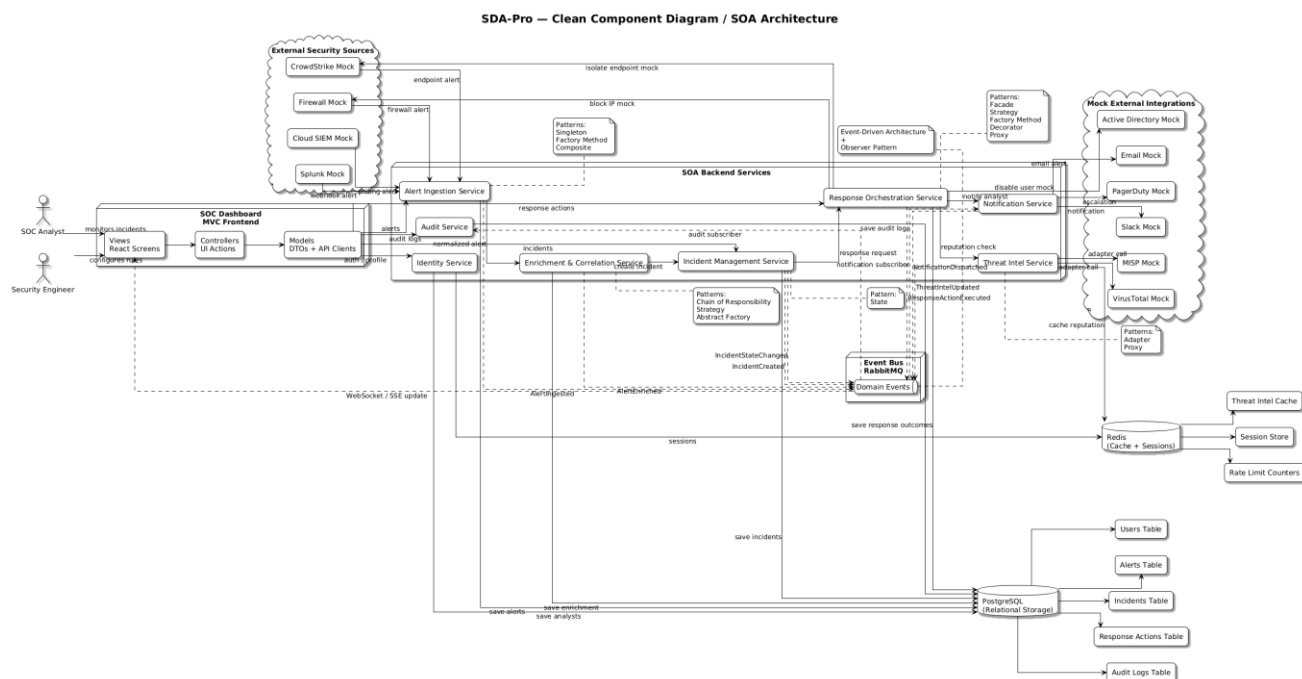


Figure 1. PlantUML component diagram showing SOA boundaries.

Component diagram explanation: This diagram proves the SOA requirement by separating the platform into named services: ingestion, enrichment, incident management, response orchestration, threat intelligence, notification, audit, identity, dashboard, and shared contracts. It also shows infrastructure dependencies: PostgreSQL for persistence, Redis for cache, RabbitMQ for events, and realtime dashboard communication. The diagram demonstrates that the project is not just a UI demo; it has explicit service responsibilities and communication paths.

Folder Structure

SDA Project/	
docker-compose.yml	
README.md	
services/alert-ingestion-service/	Spring Boot backend composition root
soc-dashboard/	React + TypeScript frontend
shared/contracts/	API/event shared contracts
docs/adr/	Architecture Decision Records
docs/api/	OpenAPI and AsyncAPI contracts
docs/plantuml codes/	PlantUML source diagrams
docs/uml diagrams/	Rendered PNG diagrams
reports/assets/	Screenshots, proof files, copied diagrams

Database Schema

Table	Purpose
analysts	Mock SOC analyst identity and role data.
alert_sources	Configured mock Splunk, Firewall, and other source definitions.
alerts	Normalized alert records and severity metadata.
alert_groups	Composite grouping of related alerts.
alert_group_members	Membership join table for alert composites.
enrichment_results	Geo, asset criticality, threat intel, and enrichment evidence.
incidents	Incident records, severity, status, and lifecycle state.
incident_alerts	Incident-to-alert correlation join table.
incident_state_transitions	History of incident state movement.
response_actions	Planned response actions such as isolate host or block IP.
response_action_outcomes	Execution results for response actions.
threat_indicators	Indicators and reputation data.
notifications	Mock Slack, PagerDuty, and Email dispatch records.
audit_logs	Security and system audit trail.

Service Boundaries

Service	Responsibility
alert-ingestion-service	Webhook/poll intake, source-specific normalization, AlertIngested event.
enrichment-correlation-service	Enrichment pipeline, threat scoring, alert grouping, incident creation.
incident-management-service	Incident CRUD and state machine transitions.
response-orchestration-service	Strategy selection, response factory, decorators, proxy execution.
threat-intel-service	Mock reputation lookup through adapters and Redis proxy cache.
notification-service	Mock Slack, PagerDuty, and Email dispatch adapters.
audit-service	Audit event creation and query API.
identity-service	Mock login/signup/profile flow for demo analysts.
soc-dashboard	Modern React dashboard and realtime monitoring screens.
shared contracts	OpenAPI, AsyncAPI, event DTOs, and shared documentation.

API Endpoints

Endpoint	Purpose
POST /ingest/webhook/{sourceType}	Accept mock Splunk/Firewall webhook alerts.
POST /ingest/poll/{sourceId}	Simulate polling an alert source.
GET /alerts, GET /alerts/{id}	List and inspect normalized alerts.
POST /enrichment/process	Run enrichment chain on an alert.
GET /incidents, POST /incidents, GET /incidents/{id}	Incident queue and incident detail support.
PATCH /incidents/{id}/state	Move incident through lifecycle states.
POST /incidents/{id}/respond	Select and execute response strategy.
POST /incidents/{id}/actions	Execute explicit response actions.
POST /threat-intel/reputation	Mock reputation lookup through adapter/proxy cache.
POST /notifications/dispatch	Mock notification dispatch.
GET /audit/events	Retrieve audit logs.
POST /auth/login, POST /auth/signup, GET /identity/me	Mock authentication and profile endpoints.
GET /dashboard/metrics, GET /dashboard/events	Dashboard metrics and realtime stream.

Event Definitions

Event	When Published
AlertIngested	A source alert is accepted and normalized.
AlertEnriched	The enrichment pipeline completes.
IncidentCreated	Correlation creates a new incident.
IncidentStateChanged	An incident moves lifecycle state.
ResponseActionExecuted	A response action finishes.
ThreatIntelUpdated	Threat reputation is looked up or cached.
NotificationDispatched	A mock notification adapter dispatches a message.
AuditLogCreated	A durable audit record is stored.

Required Demo Flow Evidence

1. Mock Splunk or Firewall alert enters /ingest/webhook/{sourceType}.
2. AlertNormalizerFactory selects a source normalizer.
3. EnrichmentChain runs geo, asset criticality, and threat intel handlers.
4. Threat intelligence is looked up through adapter classes and cached through the proxy.
5. Related alerts are grouped using the Composite pattern.
6. Correlation creates an incident in New state.
7. Incident moves through triage and containment using State pattern classes.
8. ResponseStrategy selects containment or eradication behavior.
9. ResponseActionFactory creates actions; decorators and proxies add audit and approval behavior.
10. RabbitMQ/EventBus publishes events; SSE updates dashboard panels in realtime.
11. Audit log is stored and visible in the Audit Logs screen.

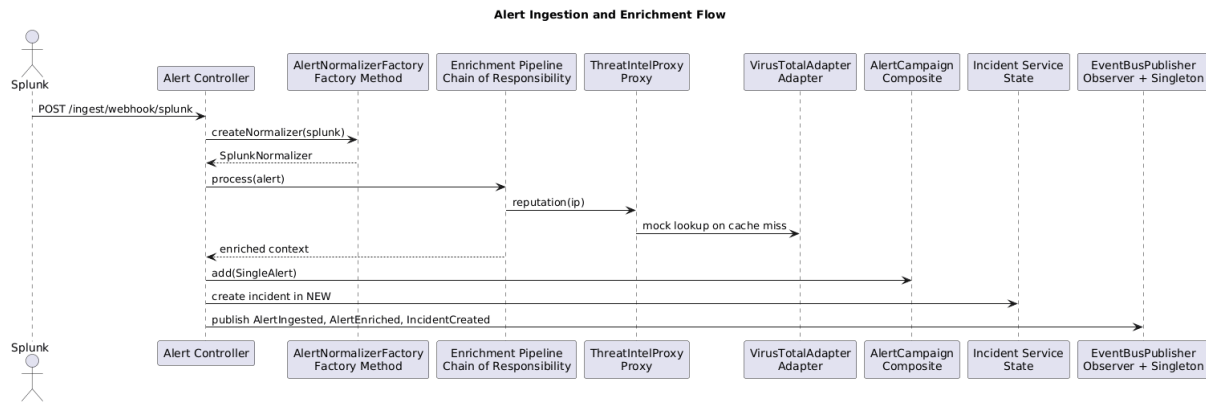


Figure 2. Alert ingestion sequence diagram.

Alert ingestion sequence explanation: This sequence maps directly to the required demo flow: a mock Splunk or Firewall alert reaches the webhook API, the normalizer factory selects a source-specific normalizer, enrichment handlers run through the chain of responsibility, threat intelligence is queried through adapter/proxy classes, an alert group is composed, an incident is created, events are published, and the dashboard/audit service receives evidence of the workflow.

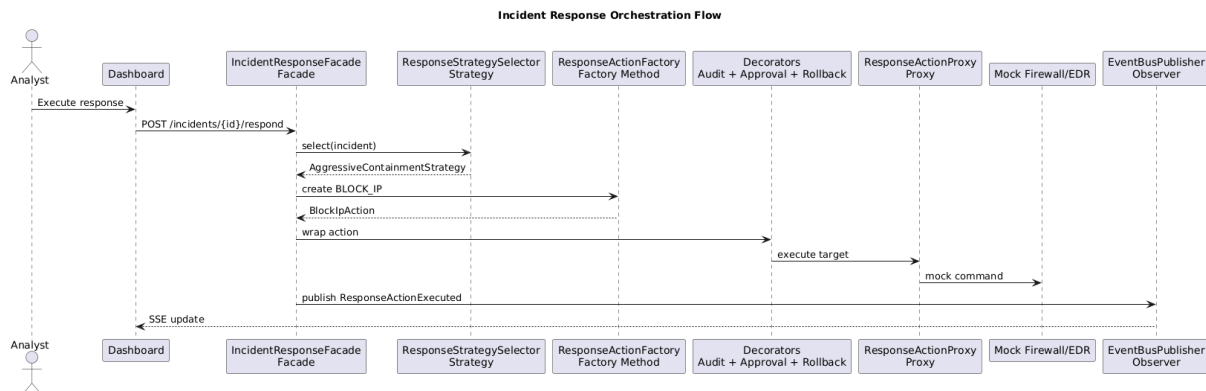


Figure 3. Incident response sequence diagram.

Incident response sequence explanation: This sequence proves the incident lifecycle and response requirements. The analyst changes incident state, the State pattern validates the transition, the strategy selector chooses the response approach, factories create actions, decorators add audit/logging behavior, proxies simulate control checks, events are published, and audit records are stored after execution.

Frontend Screens

The frontend was modernized for demo readiness: authentication is separated from the dashboard, signup is present, logout is available, raw backend identifiers and integration keys are masked, and each operational panel has a clean SOC-style layout.

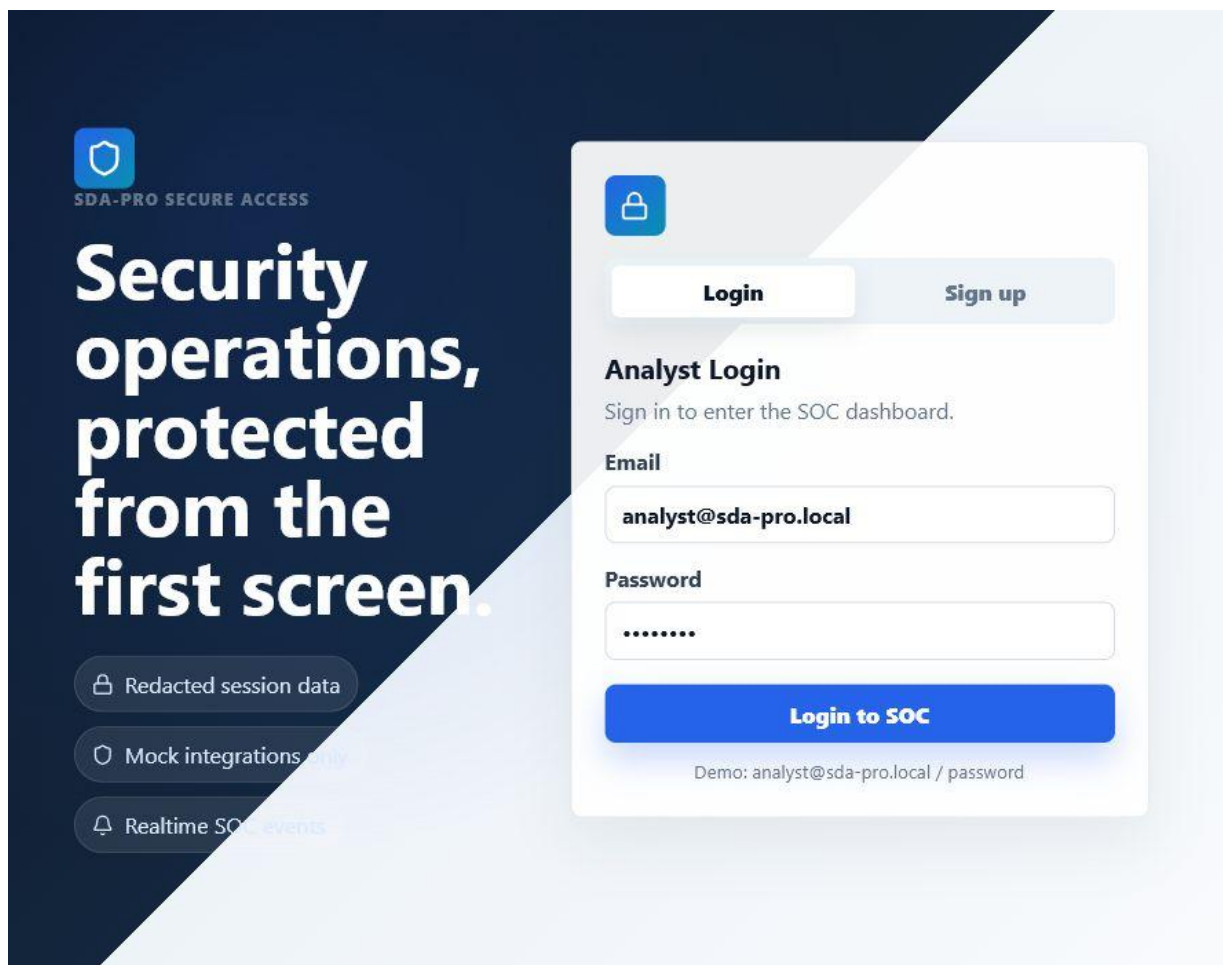


Figure 4. Login screen.

Requirement explanation: Satisfies the required Login screen and identity-service boundary. It provides a separate authentication entry point before analysts can access SOC panels. The implementation uses a mock session for demo safety instead of exposing backend tokens or integration keys in the browser.

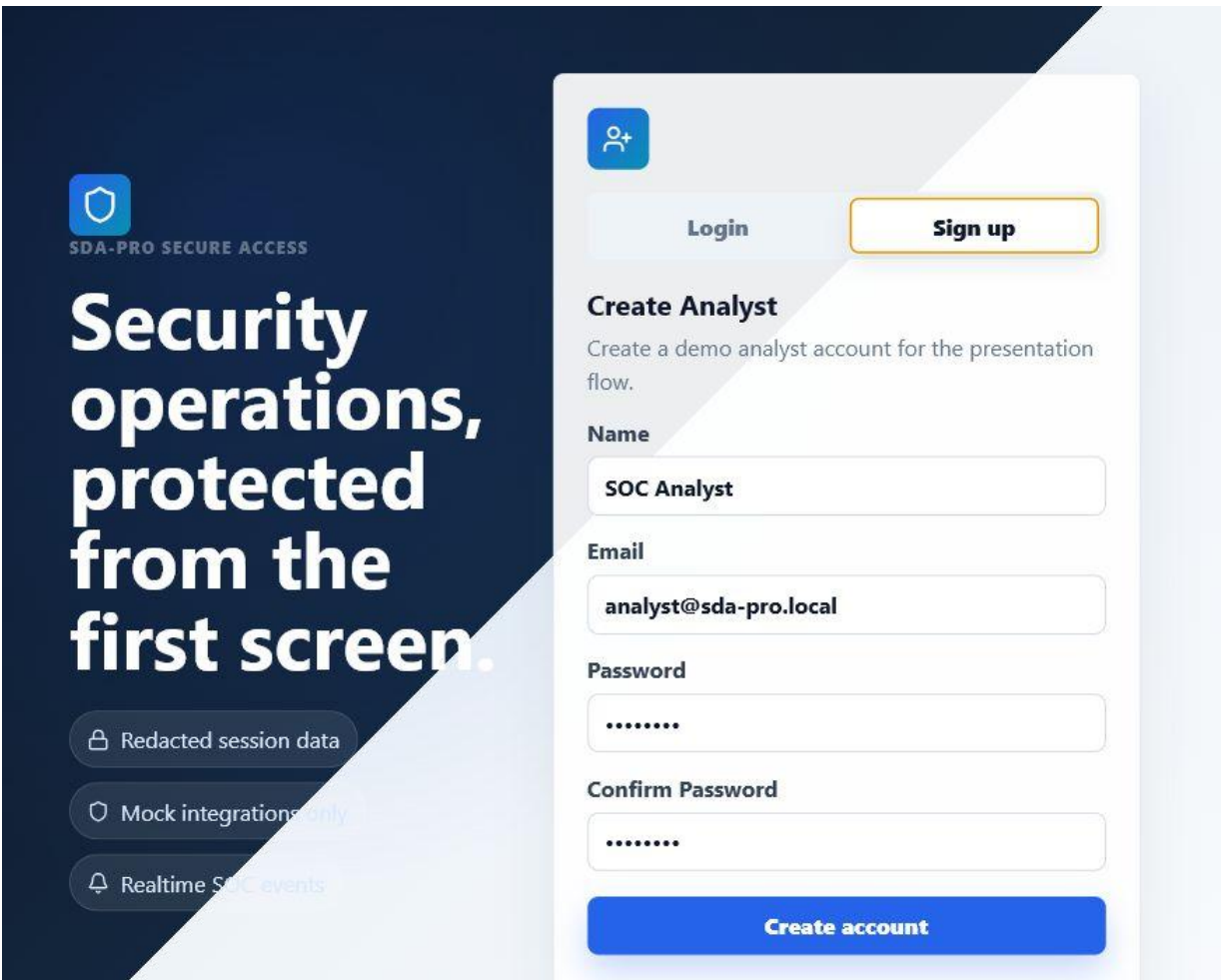


Figure 5. Signup screen.

Requirement explanation: Adds analyst onboarding support for the identity requirement. The signup view creates a mock analyst profile for demonstration and keeps the authentication experience separate from operational screens.

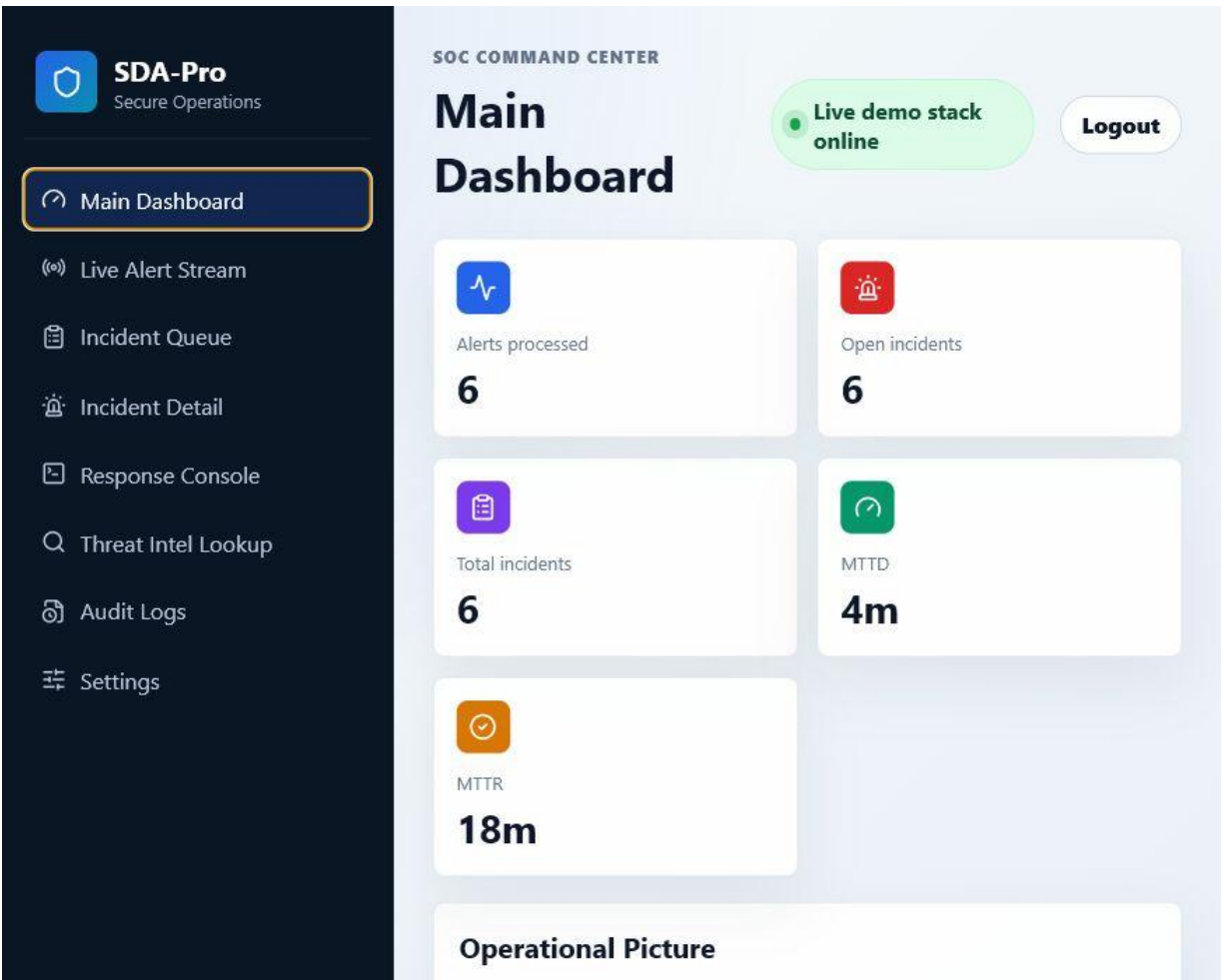


Figure 6. Main dashboard.

Requirement explanation: Satisfies the Main Dashboard requirement. It summarizes active incidents, alert volume, response outcomes, and SOC status from the dashboard metrics API so the evaluator can immediately see seeded demo data.

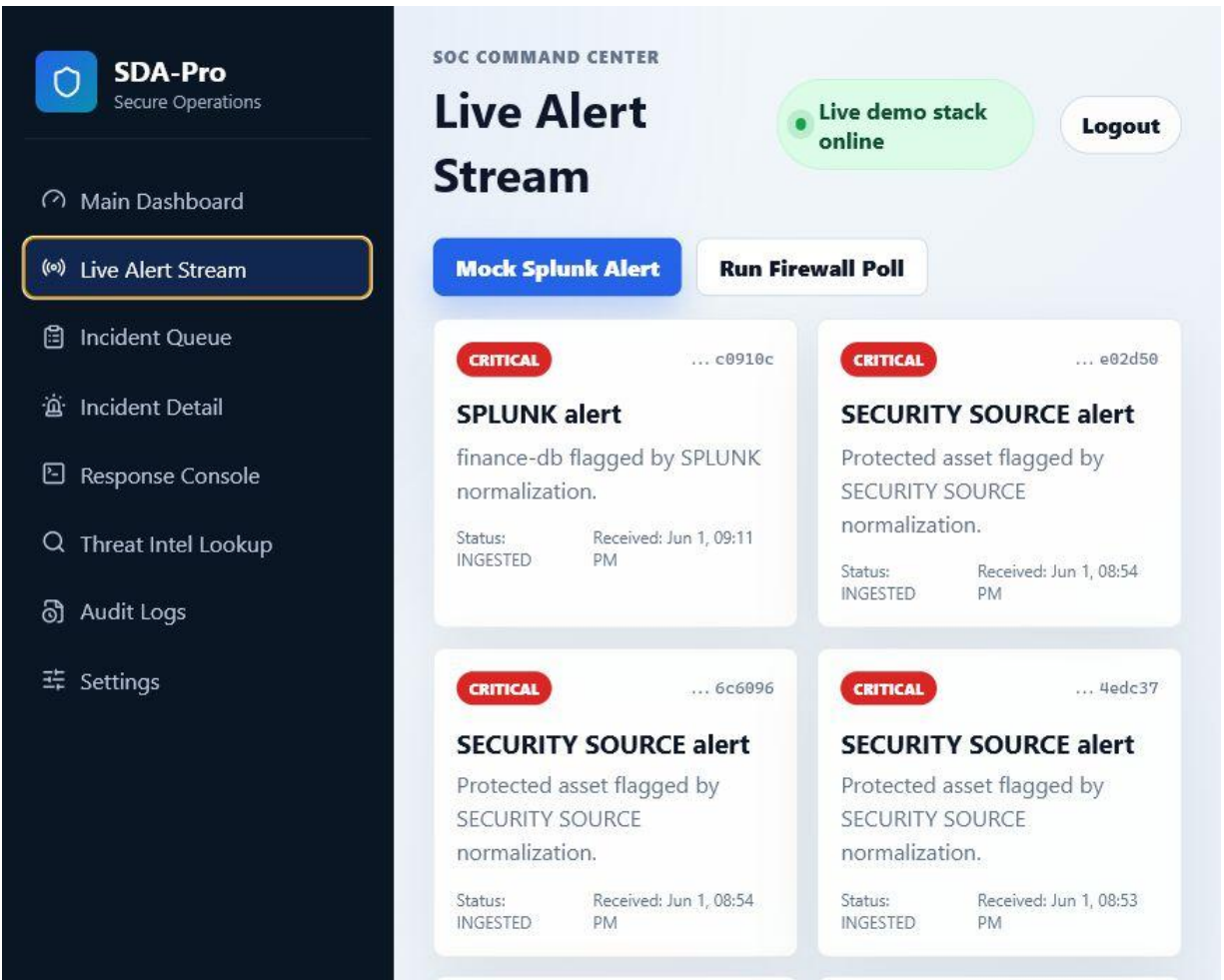


Figure 7. Live alert stream.

Requirement explanation: Satisfies the Live Alert Stream requirement. This screen demonstrates event-driven behavior and realtime push from backend events/SSE after mock Splunk or Firewall alerts enter the ingestion flow.

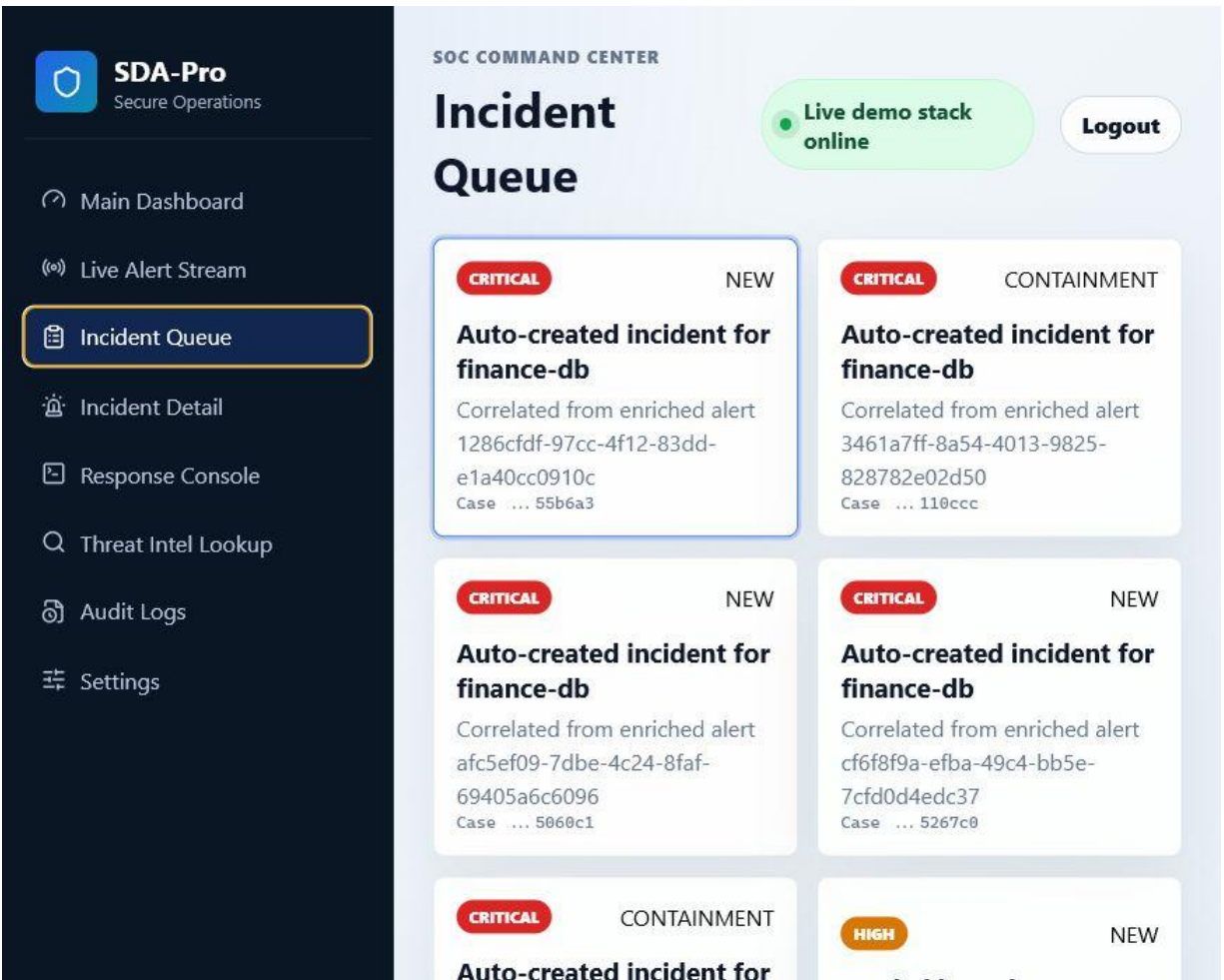


Figure 8. Incident queue.

Requirement explanation: Satisfies the Incident Queue requirement. It shows correlated incidents created from enriched alerts, supporting the required alert grouping and incident creation portion of the demo flow.

SDA-Pro
Secure Operations

SOC COMMAND CENTER

Incident Detail

Live demo stack online

Logout

Auto-created incident for finance-db

Case ...55b6a3

CRITICAL

NEW

Correlated from enriched alert 1286cdf-97cc-4f12-83dd-e1a40cc0910c

Protected Case Data

Internal IDs hidden from analyst workflow

Affected asset	finance-db
Created	Jun 1, 09:11 PM
Updated	Jun 1, 09:11 PM

Figure 9. Incident detail.

Requirement explanation: Satisfies the Incident Detail requirement. It presents incident severity, linked evidence, lifecycle state, and state transition controls, proving the State pattern and incident-management service behavior.

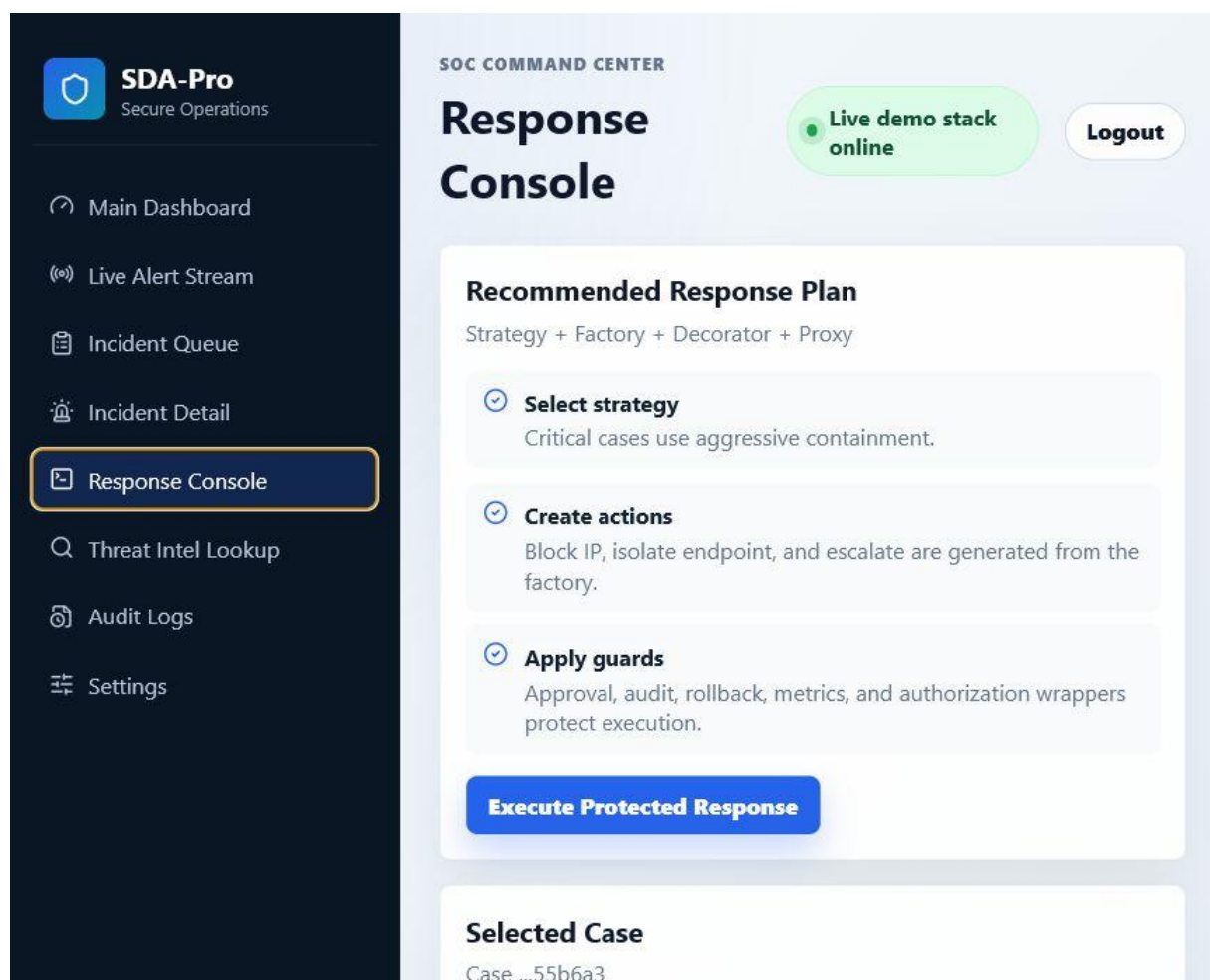


Figure 10. Response console.

Requirement explanation: Satisfies the Response Console requirement. It demonstrates response orchestration: strategy selection, factory-created actions, decorator-added audit behavior, proxy control checks, and action outcome recording.

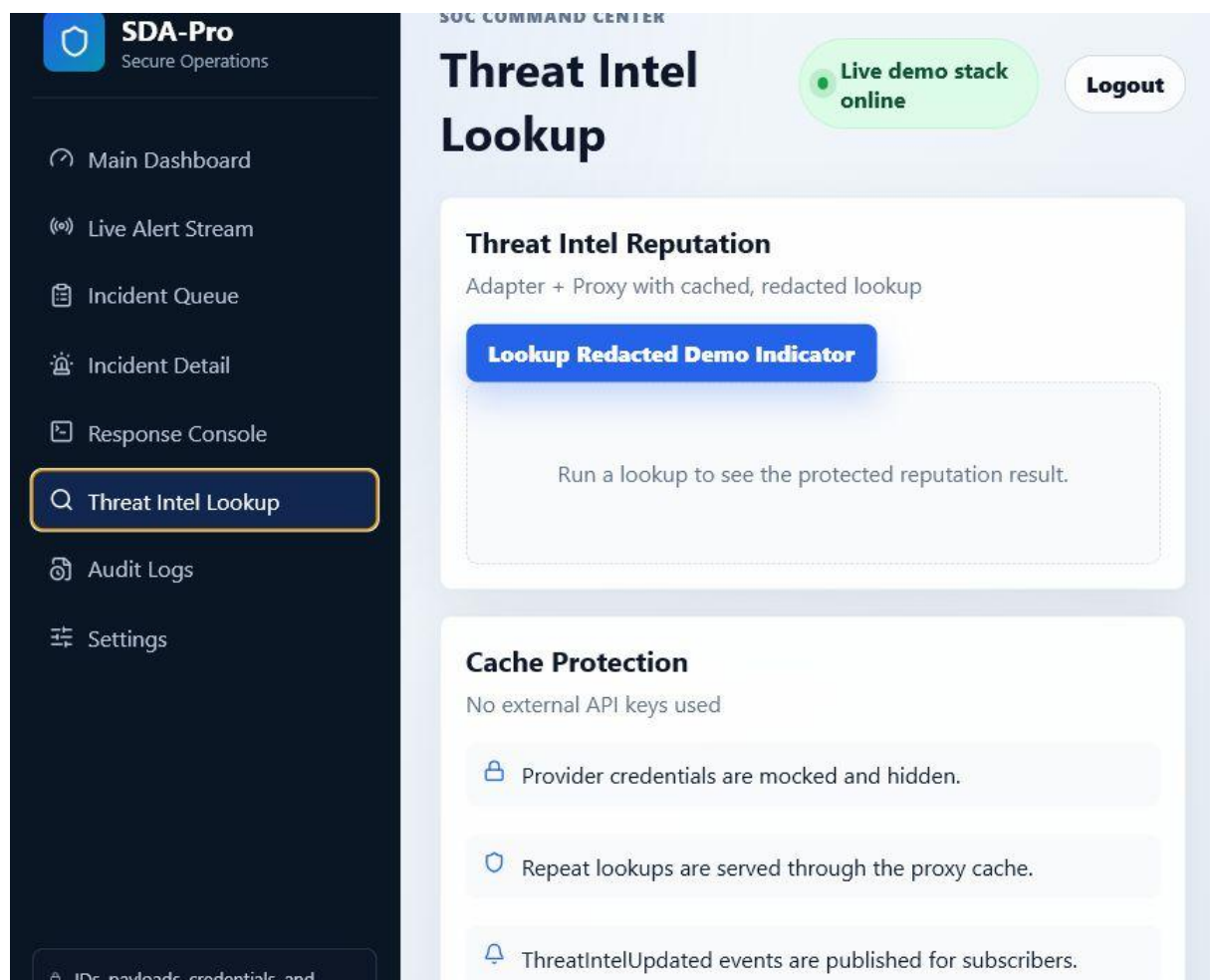


Figure 11. Threat intel lookup.

Requirement explanation: Satisfies the Threat Intel Lookup requirement. It shows mock VirusTotal/MISP style reputation lookup through adapters and the proxy cache, using Redis for repeat lookup simulation without real API keys.

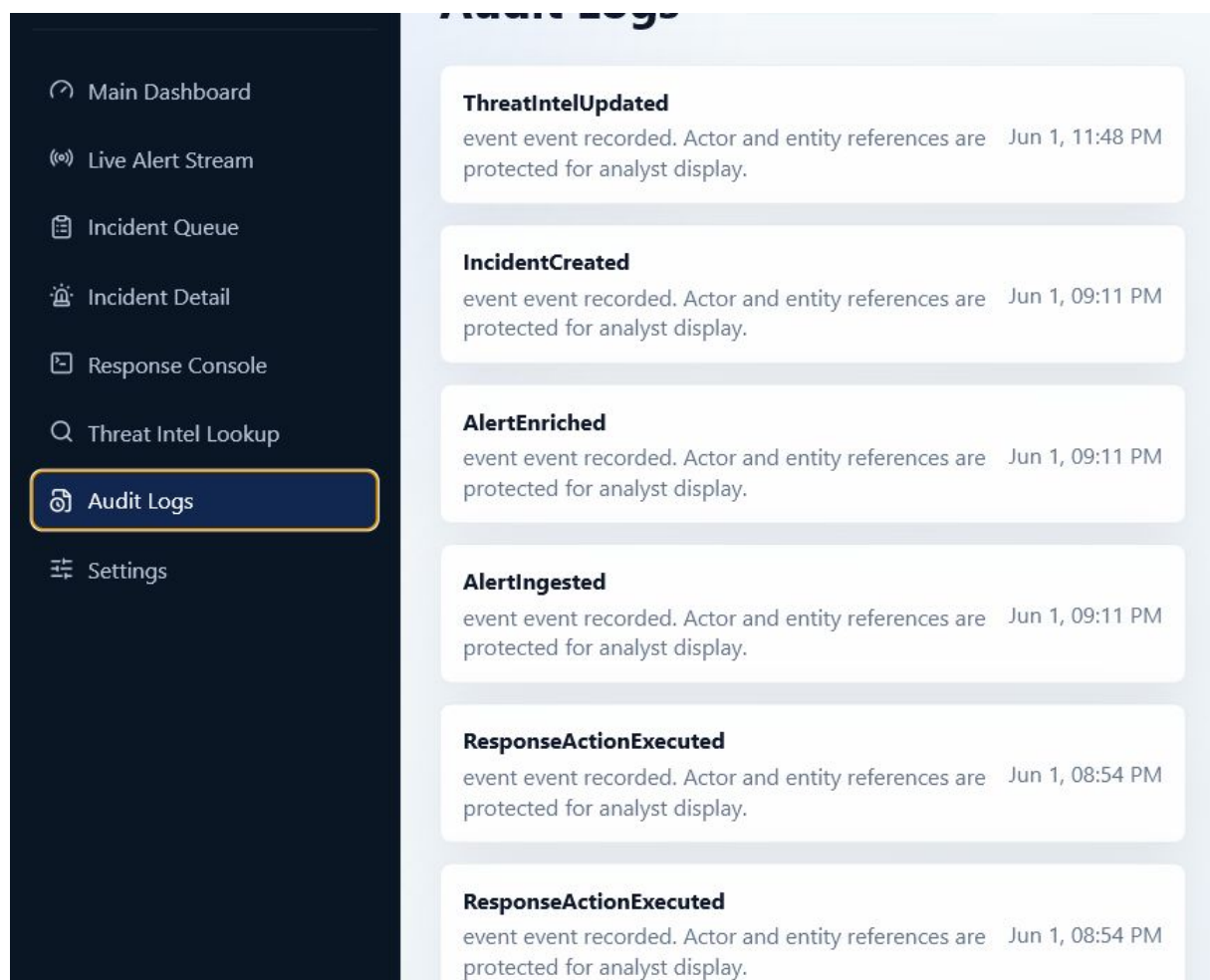


Figure 12. Audit logs.

Requirement explanation: Satisfies the Audit Logs requirement. It proves that significant system actions generate durable audit events such as alert ingestion, incident changes, response execution, notifications, and threat intel lookups.

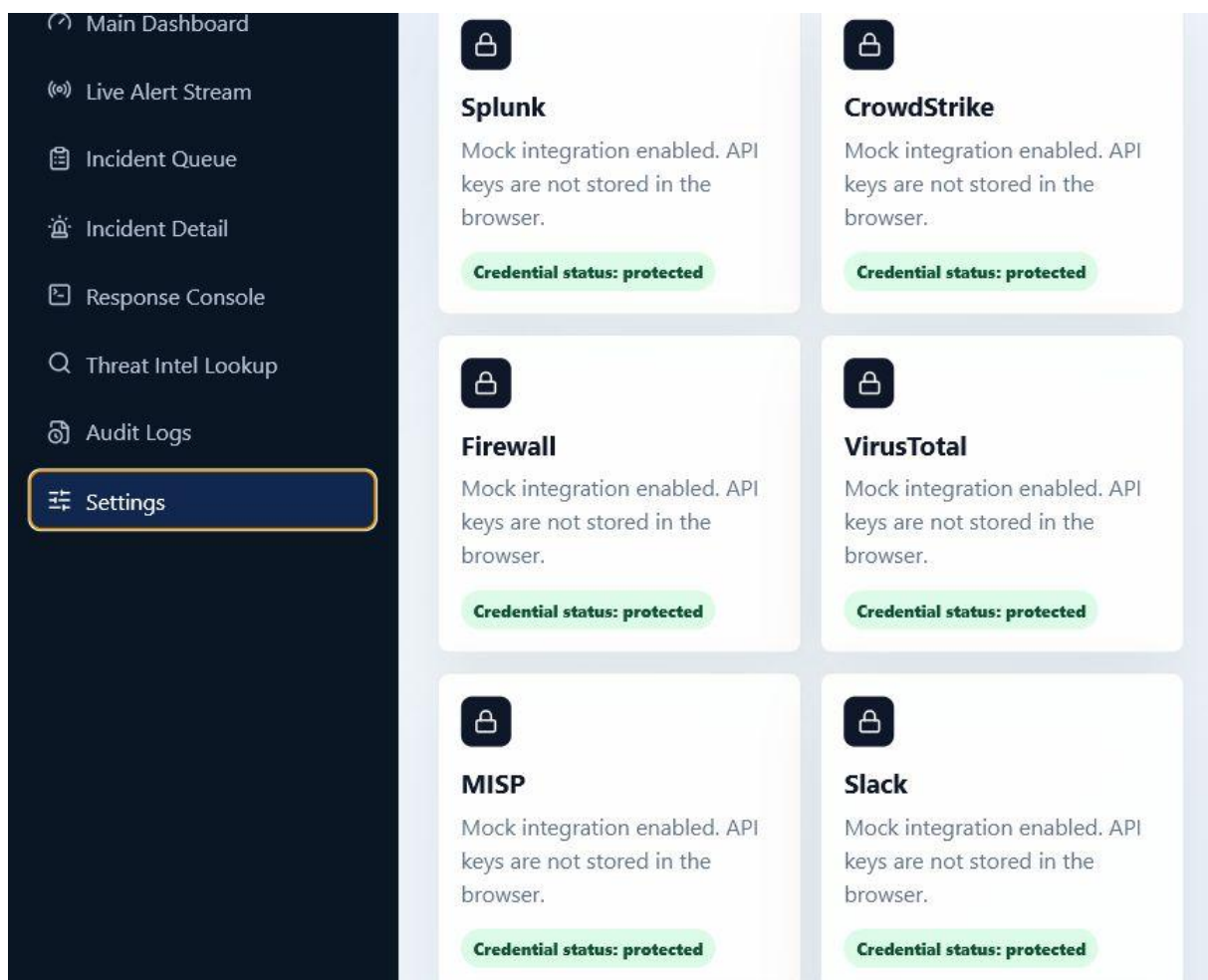


Figure 13. Settings.

Requirement explanation: Satisfies the Settings requirement and addresses security presentation. Mock integrations are visible, but credentials are shown only as protected status values; raw API keys, payload identifiers, and tokens are not displayed in analyst panels.

Design Pattern Mapping

Pattern	Implementation Evidence	Rationale
Singleton	IngestionConfigManager, EventBusPublisher	One shared configuration/event publishing component.
Factory Method	AlertNormalizerFactory, ResponseActionFactory	Create normalizers/actions without controller coupling.
Abstract Factory	EnrichmentProviderFactory, NotificationFactory	Create related provider families for mock integrations.
Composite	AlertComponent	Group alerts and individual alerts under a common interface.
Facade	IncidentResponseFacade	Simplify response orchestration into one use-

		case entry point.
Adapter	ThreatAdapters	Wrap mock VirusTotal/MISP style APIs behind one interface.
Decorator	Decorators	Add audit/logging/approval behavior around response action execution.
Proxy	ThreatIntelProxy, ResponseActionProxy	Cache lookups and simulate approval/rate-limit control.
State	IncidentState	Incident behavior changes by lifecycle phase.
Chain of Responsibility	EnrichmentHandler	Run enrichment handlers in sequence.
Observer	EventBusPublisher	Publish domain events to subscribers/dashboard.
Strategy	ResponseStrategy	Select response behavior by incident severity/context.

Code annotation proof extracted from the repository:

```

C:\Users\HP\Desktop\SDA Project\services\alert-ingestion-
service\src\main\java\edu\sdapro\domain\alert\AlertComponent.java:7: // PATTERN: Composite
C:\Users\HP\Desktop\SDA Project\services\alert-ingestion-
service\src\main\java\edu\sdapro\domain\incident\IncidentState.java:5: // PATTERN: State
C:\Users\HP\Desktop\SDA Project\services\alert-ingestion-
service\src\main\java\edu\sdapro\enrichment\EnrichmentHandler.java:7: // PATTERN: Chain of
Responsibility
C:\Users\HP\Desktop\SDA Project\services\alert-ingestion-
service\src\main\java\edu\sdapro\enrichment\EnrichmentProviderFactory.java:3: // PATTERN: Abstract
Factory
C:\Users\HP\Desktop\SDA Project\services\alert-ingestion-
service\src\main\java\edu\sdapro\events\EventBusPublisher.java:13: // PATTERN: Observer
C:\Users\HP\Desktop\SDA Project\services\alert-ingestion-
service\src\main\java\edu\sdapro\events\EventBusPublisher.java:15: // PATTERN: Singleton
C:\Users\HP\Desktop\SDA Project\services\alert-ingestion-
service\src\main\java\edu\sdapro\ingestion\AlertNormalizerFactory.java:5: // PATTERN: Factory
Method
C:\Users\HP\Desktop\SDA Project\services\alert-ingestion-
service\src\main\java\edu\sdapro\ingestion\IngestionConfigManager.java:5: // PATTERN: Singleton
C:\Users\HP\Desktop\SDA Project\services\alert-ingestion-
service\src\main\java\edu\sdapro\notification\NotificationFactory.java:3: // PATTERN: Abstract
Factory
C:\Users\HP\Desktop\SDA Project\services\alert-ingestion-
service\src\main\java\edu\sdapro\response\Decorators.java:3: // PATTERN: Decorator
C:\Users\HP\Desktop\SDA Project\services\alert-ingestion-
service\src\main\java\edu\sdapro\response\IncidentResponseFacade.java:11: // PATTERN: Facade
C:\Users\HP\Desktop\SDA Project\services\alert-ingestion-
service\src\main\java\edu\sdapro\response\ResponseActionFactory.java:3: // PATTERN: Factory Method
C:\Users\HP\Desktop\SDA Project\services\alert-ingestion-
service\src\main\java\edu\sdapro\response\ResponseActionProxy.java:3: // PATTERN: Proxy
C:\Users\HP\Desktop\SDA Project\services\alert-ingestion-
service\src\main\java\edu\sdapro\response\ResponseStrategy.java:6: // PATTERN: Strategy
C:\Users\HP\Desktop\SDA Project\services\alert-ingestion-
service\src\main\java\edu\sdapro\threat\ThreatAdapters.java:3: // PATTERN: Adapter
C:\Users\HP\Desktop\SDA Project\services\alert-ingestion-
service\src\main\java\edu\sdapro\threat\ThreatIntelProxy.java:6: // PATTERN: Proxy

```

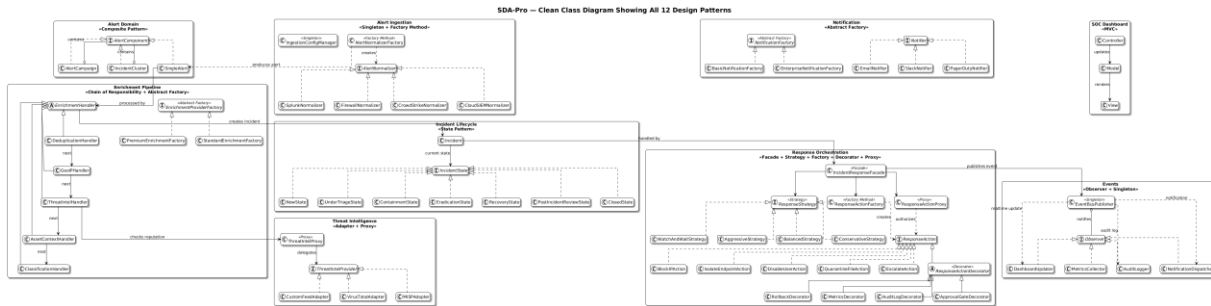



Figure 14. Class diagram showing all 12 design patterns.

Class diagram explanation: The class diagram is the primary design-pattern proof. It links the 12 required patterns to concrete classes and collaborations: factories create normalizers/actions, adapters wrap mock external products, proxies add caching and control checks, decorators add behavior around response actions, the State pattern handles incident lifecycle, the chain processes enrichment, Composite groups alerts, Strategy selects response behavior, Facade simplifies orchestration, Observer publishes events, and Singleton components centralize shared event/config access.

Architecture Style Mapping

Architecture Style	Project Mapping
SOA	Documented service boundaries for ingestion, enrichment, incident, response, threat intel, notification, audit, identity, dashboard, and contracts.
MVC	Spring controllers expose APIs, services coordinate use cases, repositories persist data; React components render views from API state.
Layered Architecture	Controller/API layer, application/service layer, domain layer, integration/adapters layer, persistence layer, infrastructure layer.
Event-Driven Architecture	RabbitMQ/EventBus events for alert ingestion, enrichment, incident creation/state changes, response execution, notification, threat intel, and audit.

Backend Proof

The backend contains Java code for controllers, domain models, design patterns, repositories, events, security demo flows, mock integrations, tests, and Docker packaging.

- Java source files found: 39 in services/alert-ingestion-service/src/main/java.
- Test classes: AuditEventCreationTest, EndToEndDemoTest, PatternTests.
- OpenAPI contract: docs/api/openapi-sdapro.yaml.
- AsyncAPI event contract: docs/api/asynccapi-events.yaml.
- ADR decisions: five ADR markdown files under docs/adr.

Frontend Proof

The frontend implements the required login, dashboard, live alert stream, incident queue/detail, response console, threat intelligence lookup, audit logs, and settings views. It masks identifiers and avoids exposing raw integration secrets.

Docker and Runtime Proof

The Docker Desktop screenshot below shows the sdaproject Compose stack running with RabbitMQ, PostgreSQL, Redis, the Spring Boot backend, and the SOC dashboard frontend. The command-line proof below was captured from the same project and lists the active backend, frontend, database, cache, and broker containers.

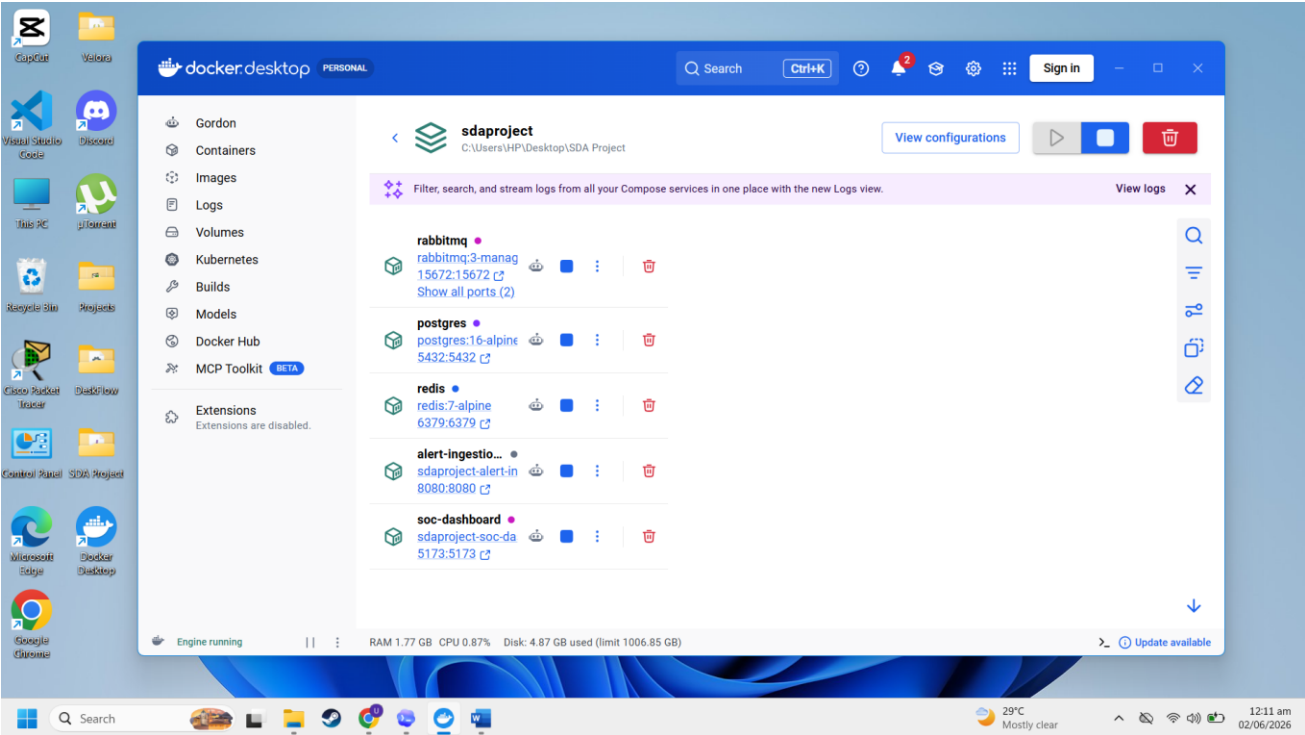


Figure 15. Docker Desktop running the SDA-Pro Compose stack.

Docker screenshot explanation: This satisfies the deployment/runtime proof requirement for docker-compose up --build. The screenshot shows the local Compose project and the running service containers needed for the demo: broker, database, cache, backend, and frontend.

NAME	IMAGE	COMMAND	SERVICE
CREATED	STATUS	PORTS	
sdaproject-alert-ingestion-service-1	sdaproject-alert-ingestion-service	"java -jar app.jar"	alert-
ingestion-service	Up 53 minutes	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp	
sdaproject-postgres-1	postgres:16-alpine	"docker-entrypoint.s..."	postgres
4 hours ago	Up 54 minutes (healthy)	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp	
sdaproject-rabbitmq-1	rabbitmq:3-management-alpine	"docker-entrypoint.s..."	rabbitmq
4 hours ago	Up 54 minutes	0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp	
sdaproject-redis-1	redis:7-alpine	"docker-entrypoint.s..."	redis
4 hours ago	Up 54 minutes	0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp	
sdaproject-soc-dashboard-1	sdaproject-soc-dashboard	"docker-entrypoint.s..."	soc-
dashboard	Up 53 minutes	0.0.0.0:5173->5173/tcp, [::]:5173->5173/tcp	

API Response Proof

Dashboard metrics API response:

```
{
  "alerts": 6,
  "openIncidents": 6,
  "mttr": "18m",
  "incidents": 6,
  "mttd": "4m"
}
```

Threat intelligence lookup API response:

```
{
  "indicator": "203.0.113.10",
  "score": 92,
  "verdict": "MALICIOUS",
  "provider": "VirusTotalMock"
}
```

Testing Evidence

Implemented test coverage includes normalizer factory, enrichment chain, incident state transitions, response strategy selection, threat intel proxy cache, audit event creation, and an end-to-end demo test.

Test set: edu.sdapro.AuditEventCreationTest

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 7.993 s -- in edu.sdapro.AuditEventCreationTest

Test set: edu.sdapro.EndToEndDemoTest

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 33.61 s -- in edu.sdapro.EndToEndDemoTest

Test set: edu.sdapro.PatternTests

Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.224 s -- in edu.sdapro.PatternTests

Required Documentation Evidence

Required Artifact	Location
README with setup/demo instructions	README.md
Docker Compose	docker-compose.yml
ADR-001 SOA vs Microservices vs Modular Monolith	docs/adr/ADR-001-soa-vs-microservices-vs-modular-monolith.md
ADR-002 Sync vs Async Communication	docs/adr/ADR-002-sync-vs-async-communication.md
ADR-003 Database Strategy	docs/adr/ADR-003-database-strategy.md
ADR-004 Threat Intel Cache Strategy	docs/adr/ADR-004-threat-intel-cache-strategy.md
ADR-005 Real-time Push Strategy	docs/adr/ADR-005-real-time-push-strategy.md
PlantUML class/component/sequence diagrams	docs/plantuml codes and docs/uml diagrams
OpenAPI and AsyncAPI	docs/api

Known Scope and Limitations

- External integrations are mocked by requirement; no real Splunk, CrowdStrike, Firewall, VirusTotal, MISP, Slack, PagerDuty, Email, or Active Directory keys are used.
- The implementation is demo-ready for SDA marking and local presentation, not hardened for production cybersecurity operations.
- Service boundaries are architecture/documentation boundaries inside a reliable local demo build, rather than independently deployed microservice artifacts for every named service.

Conclusion

The project satisfies the requested SDA-Pro deliverables: complete stack, required services, required APIs, required events, database schema, modern frontend screens, all 12 design patterns with code annotations, all 4 architecture styles, UML diagrams, ADR documentation, Docker Compose execution, seeded demo data, and automated tests.